LASO: An LLM Agent-Enabled Verifiable Semantic Oracle for Blockchain

Yijing Lin, Bowen Hua, Zhipeng Gao, Zhiqiang Xie, Junqin Huang

Abstract—Since direct access to external data and web queries can introduce nondeterminism and unverifiable dependencies into blockchain execution, blockchain oracles are widely used to bridge off-chain information to smart contracts. However, existing oracles mainly focus on structured data such as prices or binary outcomes, and cannot handle hybrid information that combines numerical feeds with unstructured textual evidence, such as market data together with related news. Meanwhile, LLM agents make it increasingly feasible to extract semantic claims from such hybrid off-chain inputs. This introduces a fundamental challenge: how to identify a trustworthy claim from multiple agent-generated hybrid semantic claims and make the selection process verifiable before the result is consumed on blockchain. In this paper, we propose LASO, an LLM agent-enabled verifiable semantic oracle for blockchain. Given hybrid off-chain inputs, LASO first employs multiple agents to independently generate semantic claims. It then uses a robust truth discovery mechanism with a dual-channel aggregation strategy to resolve conflicting multi-agent claims and select a trustworthy claim. To make this process auditable and trustworthy, LASO further encodes the claim aggregation and selection logic into zero-knowledge circuits in Circom, enabling end-to-end proofs that certify the correctness of oracle computation for on-chain verification. We implement a prototype of LASO on two representative permissioned blockchain platforms, ChainMaker and FISCO BCOS. Experimental results show that LASO achieves reliable claim selection and strong robustness under noisy and conflicting agent outputs.

Index Terms—Blockchain Oracle, LLM Agent, Truth Discovery, Zero Knowledge Proof.

1 Introduction

Blockchain was first introduced by Bitcoin [1], which leverages a decentralized ledger to maintain a tamper-resistant transaction history through a hash-chained data structure and distributed consensus. In the early Bitcoin era, blockchain systems were largely limited to simple transaction processing, with little programmability and few application scenarios. Ethereum [2] greatly expands blockchain functionality by introducing smart contracts [3], which enable general-purpose programmable logic on chain. To preserve correctness and consensus safety, blockchain execution should remain deterministic that all full nodes independently execute the same contract code and reach the same state transition. As a result, smart contracts cannot directly access external states or invoke arbitrary off-chain APIs, since such interactions would introduce nondeterministic inputs that may differ across nodes due to network delays, inconsistent responses, or evolving data [4].

Corresponding author: Zhipeng Gao and Zhiqiang Xie.
Yijing Lin is with Beijing University of Posts and Telecommunications. She is also with the Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing. Email:{yjlin}@bupt.edu.cn.
Bowen Hua, Zhipeng Gao, and Zhiqiang Xie are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. Email:{2025140850,gaozhipeng,xiezhiqiang}@bupt.edu.cn.
Junqin Huang is with the School of Computer Science (School of Cyber Science and Engineering) at Shanghai Jiao Tong University. Email:{junqin.huang@sjtu.edu.cn}.
This work is supported by National Natural Science Foundation of China (62502041, 92467203, 62372050), Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, Beijing Natural Science Foundation (L251038, L244010), CCF-Huawei Populus Grove Fund (TC202418), Fellowship of China National Postdoctoral Program for Innovative Talents (BX20240045), China Postdoctoral Science Foundation General Program (2025M773481).

Blockchain oracles address this limitation by relaying off-chain data feeds into on-chain execution. As illustrated in Fig. 1(a), oracle nodes monitor contract requests, fetch data from external sources, and process the results off chain. To avoid the single point of failure of an oracle node, practical systems use multiple oracle nodes and aggregate their reports for fault tolerance. A common approach relies on explicit endorsement, such as consensus protocols [5] or multi-signature thresholds [6], where a result is accepted only after quorum approval. Another approach uses robust statistical aggregation [7], such as the median or trimmed mean, which is more lightweight. The aggregated result is then submitted on chain for contract consumption.

However, most existing oracle designs ultimately rely on structured data feeds, such as scalar prices, index values, or binary outcomes. Because such outputs are easy to compare, aggregate, and verify, current oracle deployments are largely confined to relatively simple blockchain applications, such as DeFi price feeds and event-triggered settlement. This design choice also limits the capability boundary of oracle. It becomes increasingly insufficient for emerging agent-centric blockchain visions, where blockchains are expected not only to settle transactions, but also to discover, choose, and interact with autonomous agents across trust boundaries, as exemplified by ERC-8004 [8]. Realizing such scenarios requires oracle systems that can go beyond scalar feeds and reason over richer off-chain information.

This assumption breaks down when the underlying off-chain information takes the form of hybrid semantic claims, in which numerical values and textual evidence should be interpreted jointly. In such cases, discrepancies across sources may arise not only from numeric differences, but also from semantic variation in text, making direct align-

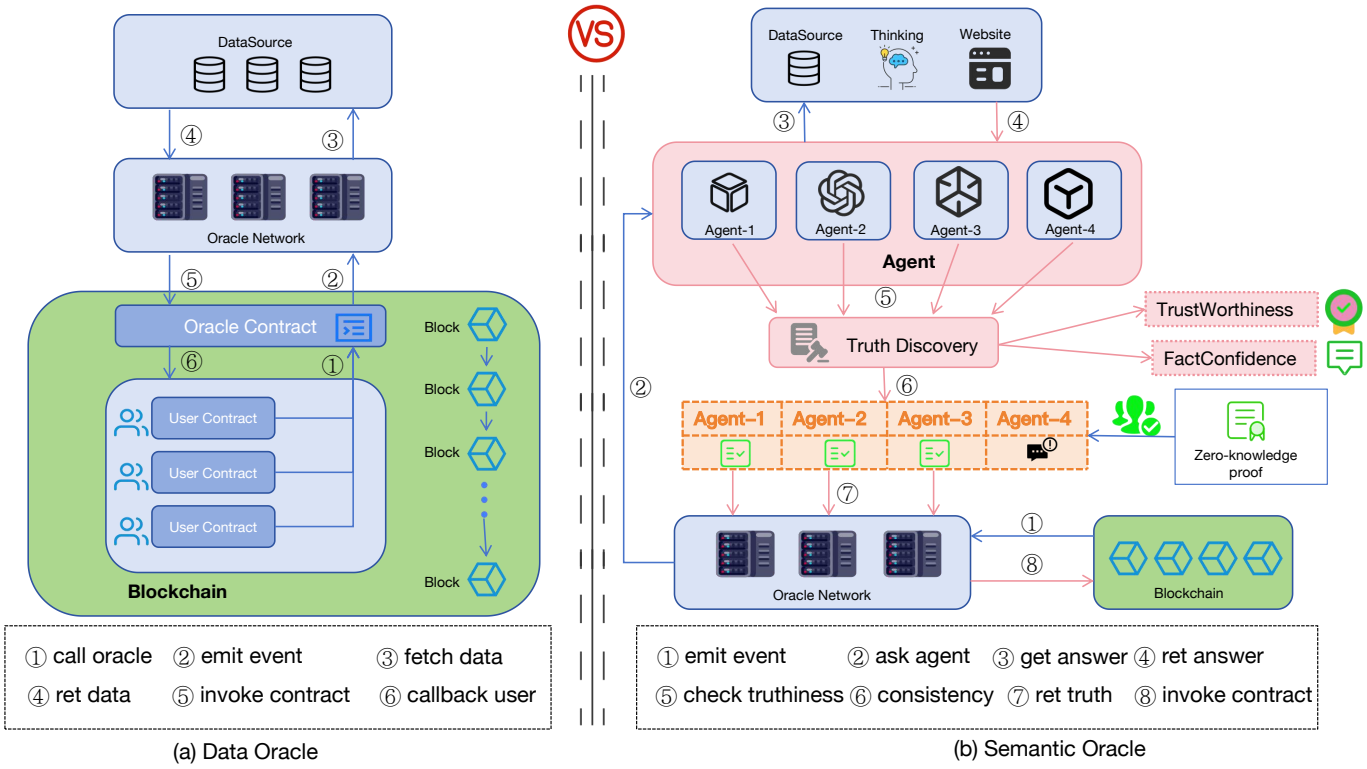


Fig. 1. Comparison of data and semantic oracle.

ment and aggregation difficult. This naturally motivates the use of LLMs, since they provide a practical way to interpret heterogeneous open-world information and convert it into comparable semantic claims. Recent studies have therefore begun to connect blockchains with LLM APIs and reconcile inconsistent LLM responses through truth discovery [9]. Nevertheless, simply invoking LLM APIs is still not sufficient for the oracle setting. What is needed is an agent-enabled oracle pipeline, where multiple agents can retrieve, interpret, and normalize hybrid off-chain information before aggregation. Even in this setting, two key challenges remain. First, existing designs do not provide a robust truth discovery mechanism for aggregating hybrid semantic claims under adversarial settings. They do not explicitly capture numerical information embedded in text, and text-embedding-based similarity may overlook important numeric changes. This problem is particularly severe when structured and unstructured evidence coexist and may be jointly manipulated. Second, the resulting semantic aggregation is still difficult to verify, since the off-chain aggregation logic is complex and cannot be directly re-executed as on-chain computation.

To address the above challenges, we propose LASO, an LLM Agent-enabled Semantic Oracle with verifiable aggregation for blockchain, given hybrid off-chain inputs consisting of structured numerical feeds and unstructured textual evidence. Specifically, as shown in Fig. 1(b), we design a robust truth discovery mechanism tailored to agent-generated hybrid semantic claims. LASO adopts a dual-channel aggregation architecture. In the numerical channel, it performs trust-aware robust truth estimation using weighted-median initialization followed by iteratively reweighted least squares (IRLS) with Tukey bisquare reweighting. In the textual channel, LASO constructs a hybrid claim affinity by combining semantic similarity with Gaussian-decayed consistency scores over numerical values extracted from text, enabling the system to distinguish semantically similar but numerically manipulated claims. Across batches, LASO further maintains source trustworthiness through a Beta-Bernoulli update rule, so that long-term reliable sources accumulate trust while persistently deviating sources are progressively gated out. To make the aggregation outcome efficiently verifiable on chain, we further implement zero-knowledge circuits in Circom that encode the claim evaluation and aggregation logic, enabling smart contracts to verify the oracle computation through succinct proofs. Following common oracle practice, we assume that a designated reporter submits the final selected claim and proof to the blockchain, while oracle-network coordination, such as leader election and quorum endorsement, is handled by existing mechanisms [4], [7], [10]. Our focus is on robust aggregation and end-to-end verifiability for LLM-agent-generated hybrid semantic claims. Our contributions are summarized as follows:

- We design LASO, a new blockchain semantic oracle framework that supports hybrid semantic claims derived by multiple LLM agents from structured numerical feeds and unstructured textual evidence, moving beyond conventional oracle outputs restricted to scalar feeds or binary outcomes.
- We propose a robust truth discovery method for agent-generated hybrid semantic claims with a dual-channel aggregation strategy to improve robustness against noisy and adversarial evidence.
- We make hybrid semantic aggregation amenable

to efficient on-chain verification by compiling the claim evaluation and aggregation logic into zero-knowledge circuits in Circom, enabling succinct proof generation without re-executing the full off-chain computation in smart contracts.
- We implement a prototype on ChainMaker and FISCO BCOS and conduct extensive experiments, demonstrating the robustness and verifiability of the proposed design.

The rest of the paper is organized as follows. Section 2 introduces the related works. Section 3 presents the proposed LLM agent-enabled semantic oracle framework for blockchain. Section 4 details the robust truth discovery mechanism for hybrid multi-agent claim aggregation. Section 5 describes the zero-knowledge verification mechanism that proves the correctness of claim aggregation. Section **??** provides the security analysis, Section 6 shows the experimental results, and Section 7 concludes the paper and outlines future work.

2 Related Works

Blockchain oracles are introduced to bridge off-chain information into on-chain execution, since smart contracts cannot directly access external states without violating deterministic execution [11]. Existing oracle designs mainly focus on delivering compact structured outputs, such as prices, index values, or binary event results, for contract consumption. Early works such as Provable (formerly Oraclize) [12] retrieve off-chain data from designated sources and provide authenticity evidence, e.g., signed attestations or TLSNotary-style proofs, to improve trust in centralized reporting. TrueBit [13] addresses a related but distinct problem by outsourcing heavy off-chain computation and using interactive dispute resolution to detect incorrect results. To tolerate faulty or adversarial reporters, practical oracle designs further employ decentralized reporting, quorum endorsement, reputation, and staking mechanisms [10].

2.1 Oracle Aggregation

In a quorum based oracle design, multiple oracle nodes independently collect off-chain data and then produce an accepted report through quorum endorsement. Z. Xu *et al.* [14] utilized verifiable-randomness-based node selection and reputation-aware scheduling to select a subset of nodes for each request and weight their responses to trigger off-chain LLM inference while preserving deterministic on-chain execution. J. Dong *et al.* [15] designed a decentralized autonomous oracle network where nodes submit signed answer blocks, and a query finalizes once blocks are collected with enough identical answers, further supported by a QoS-based reputation + noninteractive payment mechanism to prevent malicious nodes. V. Madathil *et al.* [16] designed verifiable witness encryption based on threshold signatures for a blockchain oracle that lets a payer encrypt a transaction signature such that the payee can decrypt and redeem the payment only if a set of oracles attests the real-world outcome. B. Nassirzadeh *et al.* [17] proposed a decentralized oracle network tailored for high-precision counting systems where events occur continuously and unpredictably, by assigning each proposition to a random subset of verifiers, and using majority voting to deter dishonest behaviors.

In a truth discovery based oracle design, multiple sources provide potentially conflicting reports for the same object, and the aggregator jointly estimates the latent truth and source reliability by iteratively reweighting reports. Y. Liang *et al.* [18] proposed a super-fast privacy-preserving truth discovery scheme for crowdsensing that builds the truth-discovery computation as an arithmetic circuit and realizes it via secret sharing to improve online efficiency without sacrificing reliability. J. Zhao *et al.* [19] presented a privacy-preserving truth discovery framework for continuous crowdsensing by combining homomorphic encryption with a two-server perturbation mechanism, where sensitive values are masked by random splitting across two non-colluding servers. L. Gigli *et al.* [20] designed a zero-trust oracle network for IoT applications that retrieves trusted IoT data through a blockchain-based oracle architecture. It combines a Truth Inference algorithm with a reputation system to improve data reliability and system resiliency. X. Zeng *et al.* [9] employed a truth-discovery-style aggregator to evaluate heterogeneous LLM responses, and further adopted a BFT-based consensus layer for consistency. SenFEED [21] integrated a composite truth discovery algorithm to balance latency and accuracy and designed a dynamic partially synchronous BFT protocol that supports node join and leave without system reset.

2.2 Oracle Verification

The trusted hardware-based oracle design leverages trusted execution environments (TEEs) to provide authenticated data feeds, where an enclave fetches external data and attests to the integrity of the execution and the returned results. Town Crier [22] provided an authenticated data feed for smart contracts by bridging HTTPS websites and on-chain contracts with a trusted execution environment. C. Liu *et al.* [23] proposed a TEE-based framework to extend on-chain trust to the off-chain physical world by implementing a low-cost Arm Cortex-M33 TrustZone-based monitoring device and continuously producing anti-forgery records from multiple sensors together with authenticity proofs. DEXO [24] designed a decentralized data exchange mechanism for IoT data markets that integrates TEE, secret sharing, and smart-contract-assisted fair exchange with ARM TrustZone to achieve end-to-end data confidentiality. Wukong [25] provided verifiable data feeds for smart contracts by recruiting an oracle committee to screen and authenticate data from providers, then enabling a server-side execution environment to support privacy-preserving and verifiable computation. However, TEE-based oracle designs inherit strong hardware and infrastructure assumptions. They typically rely on vendor-specific TEEs and their remote attestation services, which introduce additional trust in hardware vendors and the attestation supply chain.

In a zero-knowledge proof based oracle design, the prover first encodes the target computation as an arithmetic circuit and generates a witness from the private inputs. The prover then runs a proving algorithm to produce a succinct proof that the witness satisfies all circuit constraints. The verifier checks the proof against the public inputs

and a verification key, and accepts if the proof is valid. DECO [26] was a decentralized oracle mechanism for TLS-protected data that uses cryptographic protocols between the client and the verifier to enable verifiable extraction from a TLS session via zero-knowledge proofs. Y. Su *et al.* [27] proposed an oracle-based privacy-preserving cross-domain authentication scheme that uses an accumulator-based domain-membership proof and pseudo-identities to support mutual authentication and session-key agreement. Y. Lin *et al.* [28] utilized zero-knowledge proof to extract semantic information from off-chain data and verify data authenticity on-chain to achieve fair data exchange.

2.3 Summary

Existing oracle studies have made important progress in decentralized reporting, truth discovery, and verifiable off-chain computation. However, they still fall short of supporting the setting considered in this paper. Quorum-based oracle designs mainly certify agreement on reported outputs, but do not resolve semantic discrepancies among conflicting claims. Existing truth-discovery-based approaches either focus on numerical reports or rely primarily on semantic similarity over textual responses, and therefore do not provide a unified robust aggregation mechanism for hybrid semantic claims that combine structured numerical feeds with unstructured textual evidence. Meanwhile, existing verification-oriented oracle designs mainly verify data authenticity or outsourced computation, rather than the correctness of hybrid claim evaluation and aggregation itself. These limitations motivate the design of LASO, which unifies robust hybrid claim aggregation with zero-knowledge-based verifiability.

3 LLM Agent-Enabled Verifiable Semantic Oracle for Blockchains

Conventional data oracles mainly return structured numerical values for smart-contract consumption, while recent LLM-based oracle designs can derive richer semantic outputs from open-world evidence but often suffer from inconsistent agent-generated claims and limited verifiability of the aggregation process. In contrast, LASO produces trustworthy hybrid semantic claims from structured numerical feeds and unstructured textual evidence, together with source trustworthiness estimates derived from a specified aggregation procedure. As shown in Fig. 1(b), the proposed framework consists of five components.

Blockchain. The blockchain serves as the on-chain execution environment, where smart contracts maintain deterministic state transitions and interact with the oracle system through predefined interfaces. In our setting, on-chain contracts are divided into user contracts and oracle contracts. A user contract implements application logic, such as DeFi settlement or governance rules, and issues oracle queries when external information is required. An oracle contract acts as the on-chain interface. It records query requests and parameters, verifies submitted oracle reports and proofs, and stores the finalized oracle outputs for subsequent use by user contracts.

Oracle Network. An oracle network is an off-chain service layer composed of multiple oracle nodes $\mathcal{N}$ that collectively respond to on-chain oracle requests. It continuously monitors events emitted by the oracle contract, parses the query specification, and retrieves the corresponding hybrid off-chain results. To avoid a single point of failure and tolerate faulty or adversarial behavior, the network executes the same query across multiple nodes and later aggregates their agent-generated outputs into an accepted oracle result. Following common oracle practice, we assume that a designated reporter is selected by existing coordination mechanisms [5] [7], while our focus is on robust hybrid claim aggregation and verifiable computation. In our design, each oracle node is equipped with an LLM-based autonomous agent that retrieves specified information, interprets structured numerical feeds together with unstructured textual evidence and converts them into hybrid semantic claims, denoted by AGENTEXTRACTCLAIMS. The oracle network then invokes a truth-discovery module to evaluate conflicting claims through dual-channel aggregation and cross-batch source trustworthiness update, and finally generates a zero-knowledge proof certifying that the selected claim is computed according to the specified aggregation logic before submission to the oracle contract.

Autonomous Off-Chain Agent. The autonomous off-chain agent is an LLM-based agent deployed on each oracle node. Upon receiving an on-chain query event, the agent performs tool-assisted evidence collection and analysis, such as selecting data sources, retrieving relevant documents, filtering duplicates, and extracting structured information from heterogeneous evidence. Different from conventional data oracles that mainly relay scalar feeds or binary outcomes, the autonomous agent jointly interprets structured numerical signals and unstructured textual evidence, and converts them into hybrid semantic claims for downstream aggregation. These claims provide the standardized inputs required by the truth-discovery module and the zero-knowledge verification pipeline.

Truth Discovery. Due to stochastic generation, heterogeneous retrieval, and adversarial manipulation, different agents may produce conflicting hybrid semantic claims for the same query. To address this issue, we design a robust truth-discovery method with a dual-channel aggregation strategy for agent-generated hybrid semantic claims, denoted by TRUTHDISCOVERYAGGREGATE. It adopts a dual-channel design. In the numerical channel, it performs robust truth estimation using weighted-median initialization followed by iteratively reweighted least squares (IRLS) with Tukey bisquare weights. In the textual channel, it combines SBERT-based semantic similarity with numeric consistency extracted from text, enabling the system to distinguish semantically similar but numerically manipulated claims. Across batches, it further updates source trustworthiness through Beta-Bernoulli iterations, progressively suppressing persistently deviating or malicious sources. The details are presented in Section 4.

Zero-Knowledge Proof. Since third parties cannot directly audit whether the published oracle output is correctly computed from the specified aggregation procedure, we implement an end-to-end zero-knowledge proof mechanism for claim generation and verification, denoted by GENER-

Algorithm 1 LLM Agent-Enabled Verifiable Semantic Oracle

Require: Query q issued by the user contract to the oracle contract
Ensure: Finalized semantic claim $\mathcal{R}$ stored on-chain
1: **On-chain:** Oracle contract records q and emits an event $e(q)$
2: **Off-chain:** Oracle network observes $e(q)$ and dispatches the query to oracle nodes
3: **for all nodes in parallel do**
4: $h_o^s \leftarrow$ AGENTEXTRACTCLAIMS(q)
5: **end for**
6: $\mathcal{R} \leftarrow$ TRUTHDISCOVERYAGGREGATE$(\{h_o^s\}_{s\in\mathcal{S}})$
7: $\pi \leftarrow$ GENERATEPROOF$(\mathcal{R}, \{h_o^s\}_{s\in\mathcal{S}})$
8: **Off-chain** $\rightarrow$ **On-chain:** Reporter submits $(\mathcal{R}, \pi)$ to oracle contract
9: **if** VERIFYPROOF$(\mathcal{R}, \pi) =$ `true` **then**
10: Oracle contract accepts the submission
11: **else**
12: Oracle contract rejects the submission
13: **end if**

ATEPROOF and VERIFYPROOF. The deterministic stages of the aggregation pipeline are encoded as Circom circuits, allowing the oracle network to generate a succinct proof that the published claim and trust-related outputs are computed correctly according to the prescribed logic. This enables the oracle contract to verify the result without re-executing the full off-chain computation. The details are presented in Section 5.

Workflow. A user contract submits an oracle query to the oracle contract, which records the query metadata and emits an event. The off-chain oracle network monitors this event and dispatches the query to multiple oracle nodes. Upon receiving the query, each node's autonomous off-chain agent collects hybrid off-chain evidence and converts it into hybrid semantic claims. The oracle network then aggregates conflicting agent-generated claims using the robust truth discovery method, which jointly evaluates semantic consistency, numeric consistency, and source trustworthiness to select a trustworthy claim. To make the result publicly verifiable, the designated reporter generates a succinct zero-knowledge proof attesting that the published output is computed correctly according to the specified aggregation procedure. Finally, the reporter submits the selected claim together with the proof to the oracle contract, which verifies the proof and stores the finalized result for subsequent use by user contracts.

4 Robust Truth Discovery for Multi-Agent Claim Aggregation

Different from existing truth-discovery-based oracle methods [9], [21], [29], which either focus on numerical reports or rely primarily on textual semantic similarity, we propose a robust truth discovery method with a dual-channel aggregation strategy. It is designed for agent-generated hybrid semantic claims that combine structured numerical data with unstructured textual evidence.

4.1 Agent-generated Source Reliability Initialization

We consider an oracle network consisting of a set of nodes. Each node is equipped with an autonomous off-chain agent that follows the query-specific instructions to retrieve external information and convert heterogeneous numerical and textual inputs into standardized semantic claims for downstream aggregation. An on-chain query specified in the user contract is represented as $q = (o, \tau, \theta)$, where o denotes the target object with its designated off-chain information, τ denotes the time window or freshness constraint, and θ denotes the query-specific prompts or actions. Upon receiving the query event $e(q)$ emitted by the oracle contract, each agent collects relevant hybrid off-chain inputs within the required time window τ, and extracts semantic claims. To formalize the retrieved results produced by multiple agents, we consider a set of agent-generated sources $\mathcal{S} = \{s_1, s_2, \ldots, s_S\}$, where each $s \in \mathcal{S}$ denotes a source-side report generated by an agent according to the user-specified query q. Each event e contains a batch of target objects $\mathcal{B}_e$. For each object $o \in \mathcal{B}_e$, an agent-generated source $s \in \mathcal{S}$ may provide two types of inputs, including a numerical input $x_o^s \in \mathbb{R}$, such as an asset price, and a textual input $t_o^s \in \Sigma^*$, such as a news summary or weather description, where the text may also contain embedded numerical information. Accordingly, for each event e, the hybrid agent-generated source is denoted by $h_o^s = (x_o^s, t_o^s)$. The goal of our method is to estimate the trusted output for each queried object.

To distinguish consistently agent-generated sources from persistently deviating ones across batches, we maintain a trustworthiness score for each source using a Beta prior. Specifically, each source s is associated with two parameters (α_s, β_s), initialized as $(\alpha_0, \beta_0) = (1, 1)$, which corresponds to a uniform prior. The trustworthiness of agent-generated source s is computed as

$$r_s = \frac{\alpha_s}{\alpha_s + \beta_s}. \tag{1}$$

We adopt the Beta-Bernoulli formulation because it provides a bounded trust score in $[0, 1]$, naturally accumulates historical evidence across batches. It remains easy to interpret that α_s and β_s represent accumulated positive and negative evidence, respectively, while r_s gives the posterior estimate of source reliability.

To robustly aggregate hybrid semantic claims, we adopt a dual-channel aggregation strategy. Explicit numerical inputs are processed in the numerical channel, while unstructured textual claims are handled in the textual channel. Numerical values embedded in text are first extracted by regular expressions and incorporated into the numerical channel, and the remaining textual content is aggregated in the textual channel. The aggregation procedures of the two channels are detailed below.

4.2 Numerical channel for agent-generated sources

The numerical channel processes the numerical inputs $x_o^s \in \mathbb{R}$ provided by agent-generated sources. Its goal is to estimate a robust numerical truth $\hat{x}_o$ for each object o and to compute a numerical consistency score for each source. Before truth estimation, we first utilize a reliability-gating

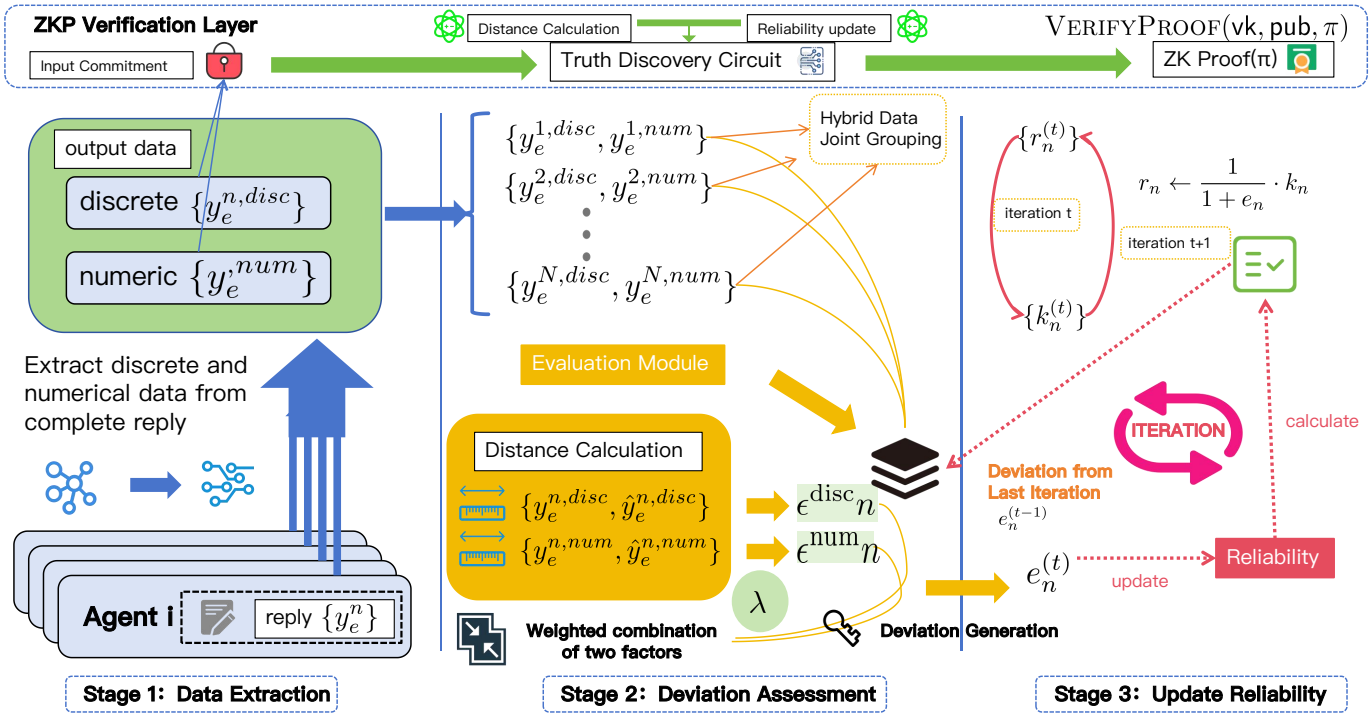


Fig. 2. Workflow. The detailed workflow includes robust truth discovery for multi-agent claim aggregation and zero-knowledge proof for correctness of semantic oracle

step to filter out sources whose historical trustworthiness is significantly below the agent group. The gating threshold is defined as gate $= \bar{r} \cdot \lambda_{\text{gate}}$, where $\bar{r}$ is the average trustworthiness of the currently participating sources and λ_{gate} is the gating factor. We retain only those sources satisfying $r_s \geq$ gate for subsequent numerical aggregation. We set $\lambda_{\text{gate}} = 0.3$ as a conservative choice, so that only sources with substantially low trustworthiness are filtered out while temporary fluctuations are less likely to remove otherwise reliable sources. This design allows the method to use historical trust information accumulated across batches to suppress clearly unreliable sources before performing batch-level estimation. We adopt a relative gate instead of a fixed threshold so that the filtering criterion can adapt to the overall trust level of the current batch.

After reliability gating, we perform robust numerical truth estimation over the retained agent-generated sources. For object o, let $\{(x_o^i, w_i)\}_{i=1}^n$ denote the retained numerical value provided by the source and their corresponding base weights after gating, where x_o^i is the numerical value provided by the i-th retained source, $w_i = r_{s_i}$ is its trustworthiness score in Eq. (1), and n is the number of retained sources after reliability gating. We first initialize the estimator by the weighted median

$$m = \arg\min_z \sum_{i=1}^{n} w_i |x_o^i - z|, \tag{2}$$

where z is a candidate center value. This provides a robust starting point against extreme values. Let $d_o^i = |x_o^i - m|, i = 1, 2, \ldots, n$, and sort $\{d_o^i\}_{i=1}^n$ in nondecreasing order as $d_o^{(1)} \leq d_o^{(2)} \leq \cdots \leq d_o^{(n)}$. Define the median absolute deviation by

$$\delta_o = \begin{cases} d_o^{\left(\frac{n+1}{2}\right)}, & n \text{ is odd}, \\ \dfrac{d_o^{\left(\frac{n}{2}\right)} + d_o^{\left(\frac{n}{2}+1\right)}}{2}, & n \text{ is even}, \end{cases} \tag{3}$$

and the corresponding robust scale estimate by

$$\sigma_o = \frac{\delta_o}{\Phi^{-1}(0.75)} \approx 1.4826\, \delta_o, \tag{4}$$

where $\Phi^{-1}(0.75) \approx 0.67449$ is the 0.75-quantile of the standard normal distribution [30].

Based on the current center and scale, we compute Tukey bisquare weights

$$u_o^i = \frac{x_o^i - m}{c \cdot \sigma_o}, \qquad w_i^{\text{Tukey}} = \begin{cases} \left(1 - (u_o^i)^2\right)^2, & |u_o^i| \leq 1, \\ 0, & |u_o^i| > 1, \end{cases} \tag{5}$$

where $c = 4.685$ is the standard robust-statistics setting [30]. The numerical truth is then updated by

$$\hat{x}_o = \frac{\sum_{i=1}^n w_i^{\text{base}} w_i^{\text{Tukey}} x_o^i}{\sum_{i=1}^n w_i^{\text{base}} w_i^{\text{Tukey}}}, \tag{6}$$

where $w_i^{\text{base}} = r_{s_i}$ denotes the source trustworthiness inherited from the cross-batch reliability model. This procedure is repeated until convergence or until a maximum number of iterations is reached. If σ_o is below a small tolerance ϵ_{tol}, which means that most retained sources report nearly identical values, we directly return the weighted median to avoid the division-by-zero issue in the Tukey bisquare weights.

After obtaining $\hat{x}_o$, we compute the numerical consistency score for each source. For the i-th retained source, the

residual is $\varepsilon_o^i = x_o^i - \hat{x}_o$. Let $\{\varepsilon_o^i\}_{i=1}^n$ be sorted in nondecreasing order of absolute value, and define the corresponding median absolute deviation by

$$\delta_o^{\text{res}} = \begin{cases} |\varepsilon_o|^{(\frac{n+1}{2})}, & n \text{ is odd}, \\ \dfrac{|\varepsilon_o|^{(\frac{n}{2})} + |\varepsilon_o|^{(\frac{n}{2}+1)}}{2}, & n \text{ is even}, \end{cases} \tag{7}$$

where $|\varepsilon_o|^{(j)}$ denotes the j-th order statistic of $\{|\varepsilon_o^i|\}_{i=1}^n$. We then define the consistency scale as

$$\gamma_o = \max\left(1.4826\,\delta_o^{\text{res}},\ \sqrt{\frac{1}{n}\sum_{i=1}^n (\varepsilon_o^i - \bar{\varepsilon}_o)^2},\ \epsilon_{\text{tol}}\right), \tag{8}$$

where

$$\bar{\varepsilon}_o = \frac{1}{n}\sum_{i=1}^n \varepsilon_o^i. \tag{9}$$

We then map the residual into a numerical consistency score by Gaussian decay:

$$c_o^{i,\text{num}} = \exp\left(-\frac{1}{2}\left(\frac{\varepsilon_o^i}{\gamma_o}\right)^2\right). \tag{10}$$

This yields a smooth score in $[0, 1]$, where reports closer to the estimated truth receive higher consistency.

4.3 Textual channel for agent-generated sources

The textual channel processes the textual inputs $t_o^s \in \Sigma^*$ provided by agent-generated sources. Since textual claims do not admit direct arithmetic aggregation, the goal of this channel is to select a trusted textual output for each object o and compute a textual consistency score for each source.

To compare textual claims, we construct an affinity matrix $\mathbf{A}_o \in \mathbb{R}^{n\times n}$ over the retained textual claims for object o. The affinity between two claims should reflect not only their semantic similarity, but also the consistency of the numerical values embedded in text. Accordingly, we first model these two signals separately and then combine them into a hybrid affinity score.

Let $\{t_o^i\}_{i=1}^n$ denote the retained textual claims for object o. Each textual claim is first encoded into a semantic vector

$$\mathbf{v}_o^i = \frac{\phi(t_o^i)}{\|\phi(t_o^i)\|_2}, \tag{11}$$

where $\phi : \Sigma^* \to \mathbb{R}^d$ denotes a sentence-level embedding function, instantiated by Sentence-BERT (SBERT) [31]. The semantic similarity between two textual claims t_o^i and t_o^j is then defined as

$$\text{sim}_o^{i,j} = (\mathbf{v}_o^i)^\top \mathbf{v}_o^j. \tag{12}$$

To capture numerical consistency embedded in text, we further extract all numerical values appearing in t_o^i by a regular-expression pattern π_{num} and denote the resulting ordered list by

$$L_o^i = \mathcal{P}(t_o^i; \pi_{\text{num}}), \tag{13}$$

where $\mathcal{P}(\cdot; \pi_{\text{num}})$ denotes the ordered extraction of all substrings matched by the pattern $\pi_{\text{num}} =$ `-?\d+(?:\.\d+)?`.

Since two textual claims may contain different numbers of numerical values and their appearance order may also differ, we first define a directional numerical consistency score from L_o^i to L_o^j. Specifically, for each extracted value $r \in L_o^i$, we find its closest numerical counterpart in L_o^j, and measure their relative deviation by Gaussian decay

$$\rho_o^{i\to j} = \frac{1}{|L_o^i|}\sum_{r\in L_o^i} \exp\left(-\frac{1}{2}\left(\frac{\min_{c\in L_o^j} \frac{|r-c|}{\max(|r|,\epsilon_{\text{tol}})}}{\tau_{\text{num}}}\right)^2\right), \tag{14}$$

where τ_{num} is the tolerance parameter for relative numerical deviation. We set $\tau_{\text{num}} = 0.02$, which corresponds to a conservative tolerance for precision-sensitive oracle scenarios. Identical numbers receive a score of 1.0, a relative deviation of 2% yields a score of approximately 0.607, and much larger deviations are rapidly suppressed toward 0.

Because the directional score is generally asymmetric, we further symmetrize it to obtain the final numerical consistency between two textual claims

$$\text{num}_o^{i,j} = \frac{1}{2}\left(\rho_o^{i\to j} + \rho_o^{j\to i}\right). \tag{15}$$

Finally, we combine semantic similarity and numerical consistency into a hybrid affinity score:

$$A_o^{i,j} = \begin{cases} \alpha\,\text{sim}_o^{i,j} + (1-\alpha)\,\text{num}_o^{i,j}, & L_o^i \neq \emptyset \text{ or } L_o^j \neq \emptyset, \\ \text{sim}_o^{i,j}, & \text{otherwise}, \end{cases} \tag{16}$$

where $\alpha \in [0, 1]$ balances semantic and numerical signals. In our implementation, we set $\alpha = 0.5$ to assign equal importance to semantic agreement and numerical consistency.

Based on the hybrid affinity matrix, we assign each textual claim a soft-voting score by accumulating support from other retained sources:

$$\text{score}_o^i = \sum_{\substack{j=1 \\ j\neq i}}^n A_o^{i,j}\, w_j, \tag{17}$$

where $w_j = r_{s_j}$ denotes the trustworthiness of the j-th retained source. The trusted textual output for object o is then selected as the claim with the highest soft-voting score,

$$\hat{t}_o = t_o^{i^\star}, \qquad i^\star = \arg\max_{1\le i\le n} \text{score}_o^i. \tag{18}$$

After selecting $\hat{t}_o$, we compute the textual consistency of each retained source with respect to this trusted output. We first evaluate semantic consistency. Let $\mathbf{v}_o^{i^\star}$ denote the normalized semantic vector of the selected textual output. For the i-th retained source, the semantic consistency is defined as

$$c_o^{i,\text{sem}} = \max\left(0, (\mathbf{v}_o^i)^\top \mathbf{v}_o^{i^\star}\right). \tag{19}$$

This clips the cosine similarity into $[0, 1]$, so that semantically closer claims receive higher semantic consistency.

We next construct a numerical reference for the values embedded in text. Let $L_o^{i^\star}$ denote the ordered numerical list extracted from the selected textual output, and let $K_o = |L_o^{i^\star}|$ be its length. Since different textual claims may contain different numbers of extracted values, we only align those retained sources whose extracted lists have the same length as $L_o^{i^\star}$. Denote the corresponding index set by $\mathcal{I}_o = \{\, i \in \{1, \ldots, n\} \mid |L_o^i| = K_o \,\}$. For each numerical position $k \in$

$\{1, \ldots, K_o\}$, we construct a weighted numerical consensus by

$$\hat{\ell}_{o,k} = \arg\min_{z} \sum_{i \in \mathcal{I}_o} w_i \left| \ell^i_{o,k} - z \right|, \tag{20}$$

where $\ell^i_{o,k}$ denotes the k-th extracted numerical value in L^i_o. This yields the numerical consensus vector $\hat{\ell}_o = (\hat{\ell}_{o,1}, \hat{\ell}_{o,2}, \ldots, \hat{\ell}_{o,K_o})$.

For each retained source $i \in \mathcal{I}_o$, we then compute the numerical residual at position k as $\varepsilon^i_{o,k} = \ell^i_{o,k} - \hat{\ell}_{o,k}$. Let $\{\varepsilon^i_{o,k}\}_{i \in \mathcal{I}_o}$ be the residuals at position k, and define the corresponding consistency scale by

$$\gamma_{o,k} = \max\left(1.4826\, \delta^{\mathrm{res}}_{o,k}, \sqrt{\frac{1}{|\mathcal{I}_o|} \sum_{i \in \mathcal{I}_o} \left(\varepsilon^i_{o,k} - \bar{\varepsilon}_{o,k} \right)^2}, \epsilon_{\mathrm{tol}} \right), \tag{21}$$

where $\delta^{\mathrm{res}}_{o,k}$ is the median absolute deviation of $\{\varepsilon^i_{o,k}\}_{i \in \mathcal{I}_o}$ and

$$\bar{\varepsilon}_{o,k} = \frac{1}{|\mathcal{I}_o|} \sum_{i \in \mathcal{I}_o} \varepsilon^i_{o,k}. \tag{22}$$

The numerical consistency at position k is then defined by Gaussian decay

$$c^{i,\mathrm{num}}_{o,k} = \exp\left(-\frac{1}{2} \left(\frac{\varepsilon^i_{o,k}}{\gamma_{o,k}} \right)^2 \right). \tag{23}$$

The overall numerical consistency of source i is obtained by averaging over all aligned positions:

$$c^{i,\mathrm{num}}_o = \frac{1}{K_o} \sum_{k=1}^{K_o} c^{i,\mathrm{num}}_{o,k}. \tag{24}$$

Finally, the textual consistency score of source i is defined by combining semantic consistency and numerical consistency:

$$c^{i,\mathrm{txt}}_o = \begin{cases} \alpha\, c^{i,\mathrm{sem}}_o + (1-\alpha)\, c^{i,\mathrm{num}}_o, & i \in \mathcal{I}_o, \\ c^{i,\mathrm{sem}}_o, & \text{otherwise}, \end{cases} \tag{25}$$

where α is the same balancing factor introduced in the hybrid affinity score in Eq. (16), so that the relative importance of semantic agreement and numerical consistency remains consistent in both claim selection and consistency evaluation. The second case is used when the extracted numerical list of source i is not aligned with that of the selected textual output. In this way, a source receives a high textual consistency score only when its claim is both semantically close to the trusted textual output and numerically consistent with its key attributes.

4.4 Agent-generated Source Reliability Update

At the end of each epoch, the numerical and textual channels produce their own consistency scores for each retained source on object o, denoted by $c^{i,\mathrm{num}}_o$ and $c^{i,\mathrm{txt}}_o$, respectively. We first combine the available channel-level scores for source s_i on object o by

$$c^i_o = \mathrm{mean}\Big(\{c^{i,\mathrm{num}}_o,\, c^{i,\mathrm{txt}}_o\} \cap \mathrm{available} \Big), \tag{26}$$

which reduces to the corresponding single-channel score when source s_i appears in only one channel.

Algorithm 2 Robust Truth Discovery for Multi-Agent Claim Aggregation

Require: Agent-generated hybrid reports $\{h^s_o\}_{s \in \mathcal{S},\, o \in \mathcal{B}_e}$
Ensure: Trusted outputs $\{\hat{x}_o, \hat{t}_o\}_{o \in \mathcal{B}_e}$ and updated trustworthiness $\{r_s\}_{s \in \mathcal{S}}$

```
Initialize or read (α_s, β_s) and compute r_s = α_s/(α_s + β_s) for all s ∈ S
for all o ∈ B_e do
    x̂_o, {c_o^{i,num}} ← NumericalChannel({x_o^s}_{s∈S})
    t̂_o, {c_o^{i,txt}} ← TextualChannel({t_o^s}_{s∈S})
end for
for all s_i ∈ S do
    Compute c_o^i from the available channel-level scores
    Aggregate {c_o^i}_{o∈B_e} into epoch-level consistency c_e^i
    α_{s_i} ← α_{s_i} + η c_e^i,   β_{s_i} ← β_{s_i} + η(1 − c_e^i)
    r_{s_i} ← α_{s_i}/(α_{s_i} + β_{s_i})
end for
return {x̂_o, t̂_o}_{o∈B_e} and updated {r_s}_{s∈S} → R
```

We then aggregate the object-level consistency scores of source s_i over the current epoch $\mathcal{B}_e$ by $c^i_e = \frac{1}{|\mathcal{O}^i_e|} \sum_{o \in \mathcal{O}^i_e} c^i_o$, where $\mathcal{O}^i_e = \{\, o \in \mathcal{B}_e \mid c^i_o \text{ is available} \,\}$.

Finally, the parameters of source s_i in Eq. (1) are updated by

$$\alpha_{s_i} \leftarrow \alpha_{s_i} + \eta\, c^i_e, \qquad \beta_{s_i} \leftarrow \beta_{s_i} + \eta\,(1 - c^i_e), \tag{27}$$

and the updated trustworthiness is recomputed by Eq. (1).

The overall workflow of the proposed robust truth discovery method is summarized as follows. For each event batch, the method first initializes or reads the historical trustworthiness of all agent-generated sources through their Beta parameters and computes the corresponding trust scores. It then performs dual-channel aggregation. In the numerical channel, the numerical reports of each object are filtered by reliability gating, and a robust numerical truth is estimated through weighted-median initialization followed by Tukey-bisquare-based reweighting. The resulting truth is further used to compute numerical consistency scores. In the textual channel, textual claims are encoded into semantic vectors, while numerical values embedded in text are extracted by regular expressions. A hybrid affinity matrix is then constructed by combining semantic similarity and numerical consistency, based on which a soft-voting procedure selects the trusted textual output. The selected textual output is further used to compute semantic and numerical consistency scores for each source. Finally, the consistency scores from the numerical and textual channels are combined to update the Beta parameters and trustworthiness of each agent-generated source. The complete procedure is summarized in Algorithm 2.

5 Zero-Knowledge Proof for Semantic Correctness

Since LASO performs dual-channel hybrid claim aggregation off-chain and publishes the resulting oracle outputs for on-chain consumption, it is necessary to verify that the published outputs are the genuine results of the specified aggregation procedure rather than arbitrary values fabricated by the designated reporter. To this end, we design

a zero-knowledge-proof-based verification mechanism that certifies the correctness of the robust truth discovery process in Section 4. Specifically, we encode the deterministic computations of the numerical channel, the textual channel, and the source-reliability update as a Circom arithmetic circuit, and generate a succinct proof attesting that the published outputs are computed correctly.

Circuit decomposition. We decompose the overall circuit into three modules:

$$C \;=\; C_{\text{NUM}} \;\wedge\; C_{\text{TXT}} \;\wedge\; C_{\text{REL}},$$

where C_{NUM} implements the numerical channel, including reliability gating, robust numerical truth estimation, and numerical consistency scoring; C_{TXT} implements the textual channel, including semantic similarity evaluation, numerical extraction from text, hybrid affinity construction, trusted textual-output selection, and textual consistency scoring; and C_{REL} implements the source-reliability update based on the channel-level consistency scores. The circuit outputs $\texttt{is_valid} = 1$ if and only if all three modules are satisfied.

Public input. For each event batch $\mathcal{B}_e$, the public input is defined as

$$\mathsf{pub} = \left(\{\hat{x}_o, \hat{t}_o\}_{o\in\mathcal{B}_e}, \{r_s\}_{s\in\mathcal{S}}\right), \tag{28}$$

where $\{\hat{x}_o, \hat{t}_o\}_{o\in\mathcal{B}_e}$ are the claimed trusted outputs and $\{r_s\}_{s\in\mathcal{S}}$ are the claimed updated trustworthiness values.

Witness. The witness is defined as

$$\begin{aligned}\mathsf{wit} = (&\{h_o^s\}_{s\in\mathcal{S},\, o\in\mathcal{B}_e}, \{m_o, \sigma_o, c_o^{i,\text{num}}\}_{o\in\mathcal{B}_e},\\ &\{\mathbf{v}_o^i, L_o^i, A_o^{i,j}, c_o^{i,\text{txt}}\}_{o\in\mathcal{B}_e}, \{c_o^i, c_e^i, \alpha_s, \beta_s\}_{s\in\mathcal{S}}),\end{aligned} \tag{29}$$

which contains the private agent-generated reports and all intermediate values required by the circuit.

Proof statement. The statement proved by the designated reporter is

$$\exists\, \mathsf{wit} \text{ s.t. } C(\mathsf{pub}, \mathsf{wit}) = 1, \tag{30}$$

which asserts that there exists a valid hidden computation trace such that the published numerical truths $\hat{x}_o$, trusted textual outputs $\hat{t}_o$, and updated source reliabilities $\{r_s\}$ are exactly the outputs produced by the dual-channel aggregation and reliability-update rules specified in Section 4.

Prove. For the fixed circuit C, the setup phase generates a key pair $(\mathsf{pk}, \mathsf{vk})$, where pk is used by the prover and vk is used by the verifier. The designated reporter constructs the witness wit and runs

$$\pi \leftarrow \textsc{GenerateProof}(\mathsf{pk}, \mathsf{pub}, \mathsf{wit}), \tag{31}$$

then submits (pub, π) to the oracle contract.

Verify. Given (pub, π), the oracle contract runs

$$b \leftarrow \textsc{VerifyProof}(\mathsf{vk}, \mathsf{pub}, \pi), \tag{32}$$

and accepts if and only if $b = \texttt{true}$. Successful verification guarantees that the submitted trusted outputs and updated source reliabilities are consistent with executing the specified numerical channel, textual channel, and reliability-update modules on the hidden witness trace.

Since Circom circuits operate over a finite field and do not support floating-point arithmetic directly, all divisions and similarity scores are implemented in fixed-point form with a global precision factor `SCALE`. In particular, weighted median, weighted averaging, Gaussian-decay-based consistency scoring, and trust updates are all mapped into field-friendly arithmetic. For textual processing, sentence embeddings and regular-expression-based number extraction are treated as off-chain preprocessing outputs and are supplied as witness inputs, while the circuit verifies that the subsequent affinity construction, voting, consistency scoring, and reliability update follow the prescribed deterministic rules.

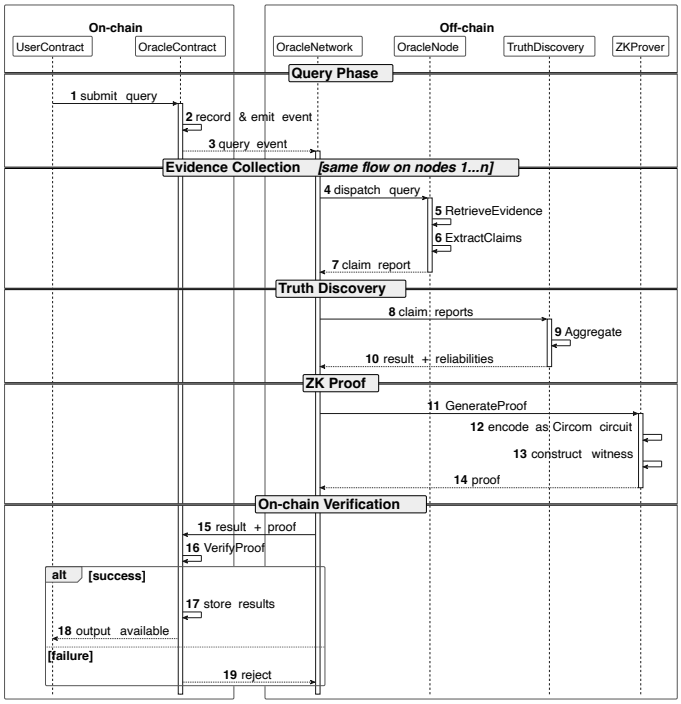


Fig. 3. Sequence diagram for the blockchain semantic oracle.

6 Experimental Results

6.1 Settings

Implementation. All experiments are conducted on a server running Ubuntu 20.04.6 LTS, equipped with two Intel Xeon Gold 6226R CPUs, providing 32 physical cores and 64 logical cores in total, 251 GiB of RAM, and four NVIDIA GeForce RTX 3090 GPUs, each with 24 GB of memory. For the blockchain component, ChainMaker v2.3.7 and the FISCO BCOS 3.0 framework are adopted to deploy the user contract and oracle contract. The oracle network connects to the blockchain network and monitors on-chain events through ports 20200 and 20201 in FISCO BCOS and ports 12301 to 12304 in ChainMaker. For agent inference, the qwen2.5:7b model is employed. Specifically, a Spring Boot service on port 8080 invokes a Flask service on port 5000, which then connects to remote hosts through SSH and calls the Ollama `/api/generate` interface on ports 11431 to 11440 to perform inference. For zero-knowledge proof generation, we implement the proving pipeline based on Circom 0.5.46, circomlib 2.0.5, and snarkjs 0.7.6. The Groth16 proving protocol over the bn128 elliptic curve is adopted as the proof backend. For on-chain verification, we implement smart contract in Solidity 0.8.20, then deploy it in the EVM environment. Figure 3 shows the detailed execution flow.

Parameters. Unless otherwise specified, we use $(\alpha_0, \beta_0) = (1, 1)$ for Beta-prior initialization, $\lambda_{\text{gate}} = 0.3$ for reliability gating, $c = 4.685$ for the Tukey bisquare weighting, $\tau_{\text{num}} = 0.02$ for relative numerical deviation in text matching, $\alpha = 0.5$ for balancing semantic and numerical signals, and $\epsilon_{\text{tol}} = 10^{-8}$ for numerical stability.

Methods. We compare our method with four representative truth discovery and oracle aggregation baselines: 1) *BasicTruth* [32], an early truth discovery method that iteratively updates fact confidence and source trustworthiness. 2) *SenteTruth* [9], which is designed for textual outputs returned by LLM-powered oracle nodes; 3) *SenFeed* [21], a decentralized oracle for streaming IoT data. 4) *DecenTruth* [29], a decentralized truth-discovering oracle for numerical reports. Since *SenFeed* and *DecenTruth* are designed for numerical data, they are evaluated only in the numerical experiments and omitted from the subsequent hybrid-claim comparisons.

Metrics. Prior LLM-oracle studies typically use Data Accuracy to evaluate whether the final aggregated output remains correct under attacks such as random response, incorrect response, and model substitution [9]. This definition is suitable for detecting clearly corrupted or strongly deviating outputs, but is less effective for hybrid semantic claims, where two reports may remain semantically close while differing in critical details. Therefore, in our textual experiments, we refine Data Accuracy (*DataAcc*) by defining a sample as correct only when its hybrid evaluation score exceeds a high-confidence threshold of 0.9, $\text{DataAcc} = \frac{1}{N}\sum_{k=1}^{N} \mathbf{1}\left[\text{score}_k \geq 0.9\right]$, where N is the number of evaluated samples and $\mathbf{1}[\cdot]$ is the indicator function. For numerical experiments, we report the Relative Root Mean Square Error (Relative RMSE), where smaller values indicate better aggregation accuracy.

6.2 Results

TODO

7 Conclusion

This paper presented an LLM-agent-enabled verifiable semantic oracle framework for blockchain, which enables smart contracts to consume trustworthy hybrid semantic claims derived from structured numerical feeds and unstructured textual evidence. At the core of the framework, we developed a robust truth discovery mechanism that jointly models semantic consistency, numeric consistency, and cross-batch source trustworthiness for multi-agent claim aggregation. We further made the aggregation process efficiently verifiable on chain by encoding the claim evaluation and aggregation logic into zero-knowledge circuits. We implemented a prototype on ChainMaker and FISCO BCOS, and experimental results showed that our method achieves strong robustness and practical verification overhead under noisy and adversarial settings. In the future, we plan to extend the framework to broader modalities and improve verification scalability.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[2] V. Buterin *et al.*, "Ethereum white paper," *GitHub repository*, vol. 1, no. 22-23, pp. 5–7, 2013.

[3] N. Szabo, "Formalizing and securing relationships on public networks," *First monday*, 1997.

[4] Y. Lin, Z. Gao, W. Shi, Q. Wang, H. Li, M. Wang, Y. Yang, and L. Rui, "A novel architecture combining oracle with decentralized learning for iiot," *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 3774–3785, 2022.

[5] L. Lamport, R. Shostak, and M. Pease, "The byzantine generals problem," in *Concurrency: the works of leslie lamport*, 2019, pp. 203–226.

[6] DOS Network, "A Decentralized Oracle Service Network to Boost Blockchain Usability with Real World Data and Computation Power," https://s3.amazonaws.com/whitepaper.dos/DOS+Network+Technical+Whitepaper.pdf, 2019, accessed: 2021-01-21.

[7] L. Breidenbach, C. Cachin, B. Chan, A. Coventry, S. Ellis, A. Juels, F. Koushanfar, A. Miller, B. Magauran, D. Moroz *et al.*, "Chainlink 2.0: Next steps in the evolution of decentralized oracle networks," *Chainlink Labs*, vol. 1, pp. 1–136, 2021.

[8] M. D. Rossi, D. Crapis, J. Ellis, and E. Reppel, "ERC-8004: Trustless Agents," https://eips.ethereum.org/EIPS/eip-8004, 2025, ethereum Improvement Proposal, Draft, created 2025-08-13.

[9] X. Zeng, Y. Xian, D. Xuan, D. Yang, C. Li, J. Chen, P. Fan, and P. Liu, "Connecting large language models with blockchain: Making smart contracts smarter," *ACM Transactions on the Web*, 2025.

[10] A. Pasdar, Y. C. Lee, and Z. Dong, "Connect api with blockchain: A survey on blockchain oracle implementation," *ACM Computing Surveys*, vol. 55, no. 10, pp. 1–39, 2023.

[11] ethereum.org, "Oracles," https://ethereum.org/en/developers/docs/oracles, 2022.

[12] provable.org, "Provable," https://provable.xyz/papers/random_datasource-rev1.pdf, 2022.

[13] J. Teutsch and C. Reitwießner, "Truebit: a scalable verification solution for blockchains," *White Papers*, 2018.

[14] Z. Xu, J. Wang, C. Zha, X. Li, and H. Yin, "Smartllm: A new oracle system for smart contracts calling large language models," in *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2023, pp. 2668–2675.

[15] J. Dong, C. Song, Y. Sun, and T. Zhang, "Daon: A decentralized autonomous oracle network to provide secure data for smart contracts," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 5920–5935, 2023.

[16] V. Madathil, S. A. Thyagarajan, D. Vasilopoulos, L. Fournier, G. Malavolta, and P. Moreno-Sanchez, "Cryptographic oracle-based conditional payments," in *NDSS 2023-Network and Distributed System Security Symposium*. Internet Society, 2023, pp. 1–8.

[17] B. Nassirzadeh, A. Heinle, S. Leonardos, A. Hasan, and V. Ganesh, "Countchain: A decentralized oracle network for counting systems," in *2024 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 2024, pp. 324–331.

[18] Y. Liang, Y. Li, and B.-S. Shin, "Fptd: Super fast privacy-preserving and reliable truth discovery for crowdsensing," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[19] J. Zhao, K. Xue, R. Li, B. Zhu, M. Li, Q. Sun, and J. Lu, "Privacy-preserving truth discovery of evolving truths for mobile crowdsensing systems," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[20] L. Gigli, I. Zyrianoff, F. Montori, L. Sciullo, C. Kamienski, and M. Di Felice, "Zonia: a zero-trust oracle system for blockchain iot applications," *IEEE Internet of Things Journal*, 2025.

[21] H. Zeng, H. Cui, C. Wang, B. Zhang, Z. Yu, and B. Guo, "Senfeed: Dynamic decentralized oracle services for accurate and real-time sensor data," in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 2025, pp. 1–10.

[22] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town crier: An authenticated data feed for smart contracts," in *Proceedings of the 2016 aCM sIGSAC conference on computer and communications security*, 2016, pp. 270–282.

[23] C. Liu, H. Guo, M. Xu, S. Wang, D. Yu, J. Yu, and X. Cheng, "Extending on-chain trust to off-chain–trustworthy blockchain data collection using trusted execution environment (tee)," *IEEE Transactions on Computers*, vol. 71, no. 12, pp. 3268–3280, 2022.

[24] Y. Li, I. Alom, W. Sun, and Y. Xiao, "Dexo: A secure and fair exchange mechanism for decentralized iot data markets," *IEEE Internet of Things Journal*, 2025.
[25] M. Li, Y. Chen, Y. Qiao, G. Ye, Z. Zhang, L. Zhu, and M. Conti, "Trust in a decentralized world: Data governance from faithful, private, verifiable, and traceable data feeds," *IEEE Transactions on Information Forensics and Security*, 2025.
[26] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, and A. Juels, "Deco: Liberating web data using decentralized oracles for tls," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1919–1938.
[27] Y. Su, Y. Wang, J. Li, Z. Su, W. Pedrycz, and Q. Hu, "Oracle based privacy-preserving cross-domain authentication scheme," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 4, pp. 602–614, 2024.
[28] Y. Lin, Z. Gao, Y. Tu, H. Du, D. Niyato, J. Kang, and H. Yang, "A blockchain-based semantic exchange framework for web 3.0 toward participatory economy," *IEEE Communications Magazine*, vol. 61, no. 8, pp. 94–100, 2023.
[29] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A decentralized truth discovery approach to the blockchain oracle problem," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.
[30] P. J. Rousseeuw and A. M. Leroy, *Robust regression and outlier detection*. John wiley & sons, 2003.
[31] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
[32] X. Yin, J. Han, and P. S. Yu, "Truth discovery with multiple conflicting information providers on the web," in *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2007, pp. 1048–1052.